

Core Microservices Design Patterns

Usage · Advantages · Disadvantages · Limitations · Real-World Examples

20 Essential Patterns | Code Examples | Pros & Cons Tables | Quick-Reference Summary

Decomposition Patterns
Data Management Patterns
Resilience Patterns
Communication Patterns
Deployment Patterns
Observability Patterns

Introduction to Microservices Design Patterns

Microservices design patterns are proven solutions to recurring architectural challenges in distributed systems. As microservices introduce complexity around decomposition, data consistency, resilience, and communication, these patterns provide structured guidance for addressing those challenges effectively.

This document covers 20 core patterns organized into 6 categories:

- Decomposition Patterns — How to split a system into services
- Data Management Patterns — How to handle data in a distributed context
- Resilience Patterns — How to prevent and handle failures
- Communication Patterns — How services talk to each other
- Deployment Patterns — How to safely release new versions
- Observability Patterns — How to monitor and debug distributed systems

Each pattern is described with:

- Intent — What problem it solves
- When to Use — Applicable scenarios
- Advantages — What you gain
- Disadvantages / Limitations — What you trade off
- Implementation Example — Code or configuration
- Real-World Usage — Where it's used in practice
- Alternatives — Other patterns to consider

Category 1: Decomposition Patterns

Pattern 1 Decompose by Business Capability [Decomposition]

Complexity: Medium

Intent: Split a monolith into microservices based on business capabilities — stable, high-level functions an organization performs (e.g., Order Management, Inventory, Payment, Shipping). Each capability maps to exactly one service.

Attribute	Details
Category	Decomposition
Complexity	Medium
Scope	System Architecture
Related To	Domain-Driven Design, Bounded Context

When to Use

- When migrating a monolith to microservices
- When business domains are well-defined and stable
- When different teams own different business functions

✓ Advantages	✗ Disadvantages / Limitations
+ Services align with org structure (Conway's Law)	- Requires deep domain knowledge to identify capabilities
+ Stable boundaries — business capabilities don't change often	- Initial decomposition can be wrong — expensive to refactor
+ Clear team ownership and accountability	- One capability may need data from another — cross-service calls
+ Enables independent scaling per business area	- Operational overhead multiplied by number of services
+ Services are loosely coupled at the business level	- May lead to chatty services if capabilities are too fine-grained

Real-World Usage

- Amazon: separate services for Cart, Orders, Recommendations, Reviews
- Netflix: Profile, Playback, Billing, Recommendation as distinct capabilities

Alternatives

- Decompose by Subdomain (DDD) — more precise using bounded contexts
- Strangler Fig — incremental extraction rather than full redesign

Pattern 2 Decompose by Subdomain (DDD) [Decomposition]

Complexity: High

Intent: Use Domain-Driven Design (DDD) to identify bounded contexts within the domain model. Each bounded context becomes a microservice with its own ubiquitous language and data model. This is the most rigorous decomposition approach.

Attribute	Details
Category	Decomposition
Complexity	High
Scope	Domain Model Design
Related To	Bounded Context, Aggregate, Anti-Corruption Layer

When to Use

- When domain complexity is high and domain experts are available
- When different parts of the domain use the same terms differently
- For greenfield projects with a well-understood business domain

✓ Advantages	✗ Disadvantages / Limitations
+ Produces highly cohesive, well-defined service boundaries	- Requires significant DDD expertise to apply correctly
+ Each service has its own model — no shared domain model pollution	- Context mapping between bounded contexts adds complexity
+ Ubiquitous language improves team communication	- Identifying correct bounded contexts is time-consuming
+ Natural fit for event-driven architectures	- Can result in too many fine-grained services
+ Services can evolve independently without impacting others	- Anti-Corruption Layer adds overhead at service boundaries

Implementation Example

```
// DDD Bounded Context: Order Context
// Order has its own model - 'Customer' means different things
// in Order context vs. Customer context

// Order Context model
public class Order {
    private OrderId id;
    private CustomerId customerId; // just a reference, not full object
    private List<OrderLine> lines;
    private Money totalAmount;
    private OrderStatus status;
}

// Anti-Corruption Layer translating from Shipping context
@Component
public class ShippingContextAdapter {
    public ShipmentRequest toShipmentRequest(Order order) {
        // Translate Order domain model to Shipping domain model
        return new ShipmentRequest(order.getId().value(),
```

```

    order.getDeliveryAddress(), order.getWeight());
}
}

```

Real-World Usage

- Large e-commerce platforms separating Catalog, Pricing, Inventory, Orders, Fulfillment

Alternatives

- Decompose by Business Capability — simpler, less rigorous

Pattern 3 Strangler Fig *[Decomposition]* Complexity: Medium

Intent: Incrementally migrate a monolith to microservices by gradually replacing specific functions with new services. Named after the strangler fig tree that grows around its host and eventually replaces it. An API Gateway routes traffic — new features go to microservices, legacy to monolith.

Attribute	Details
Category	Decomposition / Migration
Complexity	Medium
Scope	Migration Strategy
Related To	API Gateway, Anti-Corruption Layer

When to Use

- When migrating a large, complex monolith with zero downtime requirements
- When full rewrite is too risky or expensive
- When you want to incrementally validate microservices in production

 Advantages	 Disadvantages / Limitations
+ Zero downtime — monolith still handles unextracted features	- Running both systems simultaneously increases operational overhead
+ Low risk — rollback is easy (redirect traffic back to monolith)	- Dual data stores or synchronization complexity during transition
+ Incremental value delivery — extract high-value features first	- Long transition periods can lead to 'forever migration' syndrome
+ Allows learning and refinement of microservices approach	- API Gateway becomes a critical dependency
+ Works with any legacy technology stack	- Team must maintain both monolith and new services concurrently

Implementation Example

```
# Spring Cloud Gateway — Strangler Fig routing
```

```
spring:
  cloud:
    gateway:
      routes:
        # Extracted: Route to new microservice
        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**

        # Still in monolith: Route to legacy
        - id: legacy-monolith
          uri: http://legacy-monolith:8080
          predicates:
            - Path=/api/**    # catch-all for remaining routes
```



Real-World Usage

- LinkedIn migrated from a Rails monolith to microservices using this pattern
- Most large enterprises use this as the default migration strategy

 **Tip:** Extract strangled pieces feature by feature. Start with the most independent, least risky features first — e.g., authentication or email notifications.

Category 2: Data Management Patterns

Pattern 4 Database per Service [Data Management] Complexity: Medium

Intent: Each microservice owns and manages its own private database. No service can access another service's database directly — all access goes through the service's API. Services can use different database technologies suited to their needs (polyglot persistence).

Attribute	Details
Category	Data Management
Complexity	Medium
Scope	Data Architecture
Related To	Saga, CQRS, API Composition

When to Use

- This is a foundational pattern — should be used in every microservices architecture
- When different services have different data access patterns (read-heavy vs write-heavy)
- When services need to use different databases (relational, document, graph, timeseries)

✓ Advantages	✗ Disadvantages / Limitations
+ Complete service autonomy — no shared database coupling	- Distributed transactions across services are complex
+ Each service can use the optimal database type	- No foreign key relationships across service boundaries
+ Independent scaling of data stores	- Data duplication — same data may exist in multiple services
+ Schema changes in one service don't affect others	- Complex queries spanning multiple services require API Composition
+ Failure in one database doesn't directly affect others	- Operational overhead of managing many databases

Implementation Example

```
# Each service has its own data source configuration

# order-service/application.yml
spring:
  datasource:
    url: jdbc:postgresql://orders-db:5432/orders

# inventory-service/application.yml
spring:
  datasource:
    url: jdbc:mongodb://inventory-db:27017/inventory
```

```
# payment-service/application.yml
spring:
  datasource:
    url: jdbc:postgresql://payments-db:5432/payments

// Cross-service data access: ALWAYS go through API
@FeignClient(name = "inventory-service")
public interface InventoryClient {
  @GetMapping("/inventory/{sku}/stock")
  int getStock(@PathVariable String sku); // Never query DB directly
}
```

⚠ Watch Out: Never let one microservice query another service's database directly, even for performance. This creates hidden coupling and defeats the purpose of this pattern.

Pattern 5 Saga Pattern [Data Management] Complexity: High

Intent: Manage distributed transactions across multiple microservices using a sequence of local transactions. If any step fails, compensating transactions are executed to undo the work. Two implementations: Choreography (event-driven) and Orchestration (centrally coordinated).

Attribute	Details
Category	Data Management
Complexity	High
Scope	Distributed Transactions
Related To	Event Sourcing, Outbox Pattern, Kafka

When to Use

- When a business transaction spans multiple microservices
- When you need to maintain data consistency without 2PC (two-phase commit)
- E.g., placing an order requires: reserve inventory + process payment + schedule shipping

✅ Advantages	❌ Disadvantages / Limitations
+ Enables distributed transactions without 2PC lock overhead	- Compensating transactions are complex to implement correctly
+ Each service maintains its own ACID transactions	- Eventual consistency — intermediate states can be visible
+ Works with any message broker (Kafka, RabbitMQ)	- Choreography can be hard to trace and debug
+ Provides audit trail of all steps via events	- Cyclic dependencies possible in choreography approach
+ Orchestration version gives clear visibility into flow	- Requires idempotent consumers to handle duplicate messages

Implementation Example

Orchestration-based Saga with Spring + Axon Framework:

```

@Saga
public class OrderPlacementSaga {

    @StartSaga
    @SagaEventHandler(associationProperty = "orderId")
    public void on(OrderCreatedEvent event) {
        // Step 1: Reserve inventory
        commandGateway.send(new ReserveInventoryCommand(
            event.getOrderId(), event.getItems()));
    }

    @SagaEventHandler(associationProperty = "orderId")
    public void on(InventoryReservedEvent event) {
        // Step 2: Process payment
        commandGateway.send(new ProcessPaymentCommand(
            event.getOrderId(), event.getAmount()));
    }

    @SagaEventHandler(associationProperty = "orderId")
    public void on(PaymentFailedEvent event) {
        // Compensate: Release reserved inventory
        commandGateway.send(new ReleaseInventoryCommand(event.getOrderId()));
        commandGateway.send(new FailOrderCommand(event.getOrderId()));
    }

    @EndSaga
    @SagaEventHandler(associationProperty = "orderId")
    public void on(OrderCompletedEvent event) { }
}

```

Real-World Usage

- Uber Eats: order placement across restaurant, driver, payment, and mapping services
- Amazon: order workflow spanning inventory, payment, fulfillment, notification

✅ **Best Practice:** Use idempotency keys in all compensating transactions. The saga may replay messages due to failures — each step must be safe to execute multiple times.

Pattern 6 CQRS (Command Query Responsibility Segregation)

[Data Management] **Complexity: High**

Intent: Separate read (Query) and write (Command) operations into distinct models, services, and often separate databases. The write model enforces business rules; the read model is optimized for query performance. Changes flow from write to read via domain events.

Attribute	Details
Category	Data Management
Complexity	High
Scope	Data Architecture / Read-Write Separation

Related To	Event Sourcing, Database per Service, Kafka
------------	---

🔧 When to Use

- When read and write workloads have very different scaling requirements
- When queries require complex joins across many entities (use denormalized read models)
- In event-driven systems naturally — commands produce events, projections consume them
- When different teams own read vs write paths

✅ Advantages	❌ Disadvantages / Limitations
+ Read and write sides scale independently	- Significant architectural complexity
+ Read models optimized per use case (no joins, denormalized)	- Eventual consistency between write and read models
+ Write model stays clean with full business rules enforcement	- More code — separate command/query handlers, projections
+ Event history enables time-travel debugging	- Risk of stale reads during propagation delay
+ Multiple read projections from the same event stream	- Cognitive overhead for developers unfamiliar with CQRS

🏠 Implementation Example

```
// Command Side: Write Model
@CommandHandler
public void handle(CreateOrderCommand cmd) {
    // Validate, apply business rules
    Order order = new Order(cmd.getOrderid(), cmd.getItems());
    orderRepository.save(order);
    eventBus.publish(new OrderCreatedEvent(order));
}

// Query Side: Read Model (projection/view)
@EventHandler
public void on(OrderCreatedEvent event) {
    // Build denormalized view for fast reads
    OrderSummaryView view = new OrderSummaryView(
        event.getOrderid(),
        event.getCustomerName(), // pre-joined
        event.getTotalAmount(),
        event.getStatus()
    );
    orderSummaryRepository.save(view); // Could be Elasticsearch
}

// Query handler — no joins needed
@QueryHandler
public List<OrderSummaryView> handle(GetOrdersQuery query) {
    return orderSummaryRepository.findByCustomerId(query.getCustomerId());
}
```

🏢 Real-World Usage

- Microsoft uses CQRS extensively in Azure cloud services
- Axon Framework provides full CQRS + Event Sourcing infrastructure for Spring Boot

Pattern 7 Event Sourcing [Data Management] **Complexity: High**

Intent: Store the state of an entity as a sequence of immutable events rather than its current state. To get the current state, replay all events. Every state transition is captured as a named, timestamped domain event in an append-only event store.

Attribute	Details
Category	Data Management
Complexity	High
Scope	State Persistence
Related To	CQRS, Saga, Domain Events

When to Use

- When a full audit trail of all state changes is required (finance, healthcare, compliance)
- When you need to time-travel — see the state of an entity at any point in time
- In conjunction with CQRS for building multiple read projections

✓ Advantages	✗ Disadvantages / Limitations
+ Complete, immutable audit log — every state change recorded	- Query complexity — must replay events to get current state (use snapshots)
+ Time-travel: reconstruct state at any point in time	- Schema evolution of events is complex and must be versioned
+ Events are the source of truth — never lose history	- High storage requirements as event history grows
+ Enables multiple read projections from same event stream	- Eventual consistency for read projections
+ Debugging by replaying events to reproduce issues	- Steep learning curve — unfamiliar paradigm for most developers

Implementation Example

```
// Events stored in event store (append only)
OrderCreatedEvent { orderId, items, timestamp }
ItemAddedEvent    { orderId, item, timestamp }
PaymentProcessed  { orderId, amount, timestamp }
OrderShippedEvent { orderId, trackingId, timestamp }

// Reconstruct current state by replaying events
@Aggregate
public class Order {
    private OrderId id;
    private List<OrderLine> lines = new ArrayList<>();
```

```

private OrderStatus status;

@EventSourcingHandler
public void on(OrderCreatedEvent e) {
    this.id = e.getOrderId();
    this.status = OrderStatus.CREATED;
}

@EventSourcingHandler
public void on(PaymentProcessedEvent e) {
    this.status = OrderStatus.PAID;
}
}

// Snapshot for performance (avoid replaying 10,000 events)
// Take snapshot every 100 events

```

 **Tip:** Use event snapshots to avoid replaying thousands of events. Take a snapshot of current state every N events (e.g., every 100) and replay only from the last snapshot.

Pattern 8 Transactional Outbox [Data Management] **Complexity: Medium**

Intent: Solve the dual-write problem — ensuring both the database write and the message publish succeed atomically. Write the event to an 'outbox' table in the same local database transaction as the business data, then a relay process publishes outbox events to the message broker.

Attribute	Details
Category	Data Management
Complexity	Medium
Scope	Event Publishing Reliability
Related To	Saga, Event Sourcing, Change Data Capture

When to Use

- Whenever you need guaranteed event publishing alongside a database write
- When you cannot afford to lose events (financial transactions, order events)
- When using at-least-once delivery with message brokers

✓ Advantages	✗ Disadvantages / Limitations
+ Atomic — DB write and event publish succeed or fail together	- Polling the outbox table adds latency (vs direct publish)
+ No lost events even if message broker is temporarily unavailable	- Duplicate events possible (at-least-once) — consumers must be idempotent
+ Enables at-least-once delivery guarantee	- Additional outbox table and relay process to maintain
+ Works with any relational database	- Higher write load on the database
+ Decouples business logic from messaging	- CDC tooling (Debezium) adds operational

infrastructure

complexity

Implementation Example

```
// Single database transaction: save order + save outbox event
@Transactional
public Order createOrder(CreateOrderRequest req) {
    Order order = orderRepository.save(new Order(req));

    // Save to outbox IN THE SAME TRANSACTION
    OutboxEvent outboxEvent = new OutboxEvent(
        UUID.randomUUID(),
        "order-events",          // Kafka topic
        "ORDER CREATED",
        objectMapper.writeValueAsString(order),
        Instant.now()
    );
    outboxRepository.save(outboxEvent);
    return order; // Transaction commits both atomically
}

// Debezium CDC reads outbox table changes and publishes to Kafka
// OR a scheduled polling relay:
@Scheduled(fixedDelay = 1000)
public void relayOutboxEvents() {
    List<OutboxEvent> pending = outboxRepository.findPending(50);
    pending.forEach(e -> {
        kafkaTemplate.send(e.getTopic(), e.getPayload());
        outboxRepository.markPublished(e.getId());
    });
}
```

Category 3: Resilience Patterns

Pattern 9 Circuit Breaker [Resilience] Complexity: Medium

Intent: Prevent cascading failures by detecting failures in downstream services and short-circuiting calls to them. Like an electrical circuit breaker — when failures exceed a threshold, the circuit 'opens' and requests fail fast without calling the failing service.

Attribute	Details
Category	Resilience
Complexity	Medium
Scope	Fault Tolerance
Related To	Retry, Bulkhead, Fallback, Resilience4j
States	CLOSED (normal) → OPEN (failing) → HALF-OPEN (testing)

When to Use

- Any synchronous inter-service call that could fail or become slow
- When a downstream service is known to be flaky or have SLA issues
- When you need fast-fail behavior instead of thread exhaustion from timeouts

✓ Advantages	✗ Disadvantages / Limitations
+ Prevents cascading failures — isolates faults at service boundaries	- False positives — transient network issues may open the circuit unnecessarily
+ Fast failure — no waiting for timeout when circuit is open	- Half-open state requires careful tuning to avoid flip-flopping
+ Gives failing service time to recover	- Adds complexity to service call logic
+ Metrics on open/close events provide valuable insights	- Requires fallback strategy to be useful
+ Configurable per downstream service	- Circuit state is per-instance — not cluster-wide by default

Implementation Example

```
# Resilience4j Circuit Breaker configuration
resilience4j:
  circuitbreaker:
    instances:
      payment-service:
        failure-rate-threshold: 50      # Open at 50% failure rate
        slow-call-rate-threshold: 80   # Also consider slow calls
        slow-call-duration-threshold: 2s # 2s = slow
        wait-duration-in-open-state: 15s # Wait before HALF-OPEN
        sliding-window-size: 20         # Track last 20 calls
        permitted-calls-in-half-open: 3 # Test with 3 calls
```

```
@Service
public class PaymentService {
    @CircuitBreaker(name = "payment-service", fallbackMethod = "paymentFallback")
    public PaymentResponse charge(PaymentRequest req) {
        return paymentClient.charge(req);
    }

    private PaymentResponse paymentFallback(PaymentRequest req, Exception ex) {
        // Queue payment for retry later, return pending status
        asyncPaymentQueue.enqueue(req);
        return PaymentResponse.pending(req.getOrderId());
    }
}
```

Pattern 10 Bulkhead [Resilience] **Complexity: Medium**

Intent: Isolate critical resources (thread pools, connection pools) into separate pools per downstream service. Named after ship bulkheads that prevent one flooded compartment from sinking the whole ship. Prevents a slow/failing service from consuming all resources and starving other services.

Attribute	Details
Category	Resilience
Complexity	Medium
Scope	Resource Isolation
Related To	Circuit Breaker, Retry, Thread Pool
Types	Thread Pool Bulkhead / Semaphore Bulkhead

When to Use

- When multiple downstream services are called and one is slow
- When you must guarantee availability of critical paths (e.g., checkout must not be impacted by recommendations)
- In high-traffic services with multiple external dependencies

✓ Advantages	✗ Disadvantages / Limitations
+ Prevents resource starvation — one slow service can't exhaust all threads	- Increases total thread count — more memory overhead
+ Improves overall system stability under partial failure	- Complex tuning — wrong pool sizes cause under/over-utilization
+ Independent resource tuning per downstream service	- Semaphore bulkhead doesn't protect against slow I/O blocking threads
+ Pairs powerfully with Circuit Breaker for full fault isolation	- Per-instance isolation — doesn't help with cluster-level issues
+ Thread pool variant enables non-blocking with <code>CompletableFuture</code>	- Adds configuration complexity

Implementation Example

```
resilience4j:
  thread-pool-bulkhead:
    instances:
      inventory-service:      # Critical path
        core-thread-pool-size: 10
        max-thread-pool-size: 20
        queue-capacity: 50
      recommendation-service: # Non-critical, smaller pool
        core-thread-pool-size: 3
        max-thread-pool-size: 5
        queue-capacity: 10

@ThreadPoolBulkhead(name = "inventory-service")
public CompletableFuture<InventoryStatus> checkStock(String sku) {
    return CompletableFuture.supplyAsync(
        () -> inventoryClient.getStock(sku));
}
```

Pattern 11 Retry with Exponential Backoff [Resilience] **Complexity:** Low

Intent: Automatically retry failed operations, waiting progressively longer between attempts (exponential backoff) and adding random jitter to avoid synchronized retry storms (thundering herd problem).

Attribute	Details
Category	Resilience
Complexity	Low
Scope	Transient Fault Handling
Related To	Circuit Breaker, Idempotency

When to Use

- For transient failures: network blips, temporary service unavailability
- When calling external APIs or cloud services prone to throttling
- Only for idempotent operations — retrying non-idempotent operations causes duplicates

✓ Advantages	✗ Disadvantages / Limitations
+ Handles transient failures transparently	- Can mask underlying reliability issues if overused
+ Simple to implement with Resilience4j or Spring Retry	- Must NOT retry non-idempotent operations (POST, charge payment)
+ Exponential backoff + jitter reduces load during recovery	- Retries increase total latency for the caller
+ Configurable per service / operation	- Without jitter, all instances retry

+ Pairs well with Circuit Breaker (don't retry when circuit is open)

simultaneously causing spikes

- Circuit Breaker must wrap retries to avoid retrying into an open circuit

Implementation Example

```
resilience4j:
  retry:
    instances:
      inventory-service:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2 # 500ms, 1s, 2s
        randomized-wait-factor: 0.3 # +/- 30% jitter
        retry-exceptions:
          - java.net.ConnectException
          - java.io.IOException
        ignore-exceptions:
          - com.example.BusinessException # Don't retry business errors

@Retry(name = "inventory-service", fallbackMethod = "retryFallback")
@CircuitBreaker(name = "inventory-service") # Circuit breaker wraps retry
public StockResult checkStock(String sku) {
    return inventoryClient.checkStock(sku);
}
```

✅ **Best Practice:** Apply Circuit Breaker outermost, then Retry, then Bulkhead innermost. Circuit Breaker prevents retrying into an open circuit, saving resources.

Category 4: Communication Patterns

Pattern 12 API Gateway [Communication] Complexity: Medium

Intent: Provide a single entry point for all external client requests. The gateway routes requests to appropriate backend services, handles cross-cutting concerns (auth, rate limiting, SSL), and aggregates responses when needed. Clients interact only with the gateway — never directly with services.

Attribute	Details
Category	Communication
Complexity	Medium
Scope	External Access Management
Related To	BFF, Service Discovery, Circuit Breaker
Tools	Spring Cloud Gateway, Kong, AWS API Gateway, Nginx

When to Use

- In every microservices system — this is a standard foundational pattern
- When you need to enforce authentication/authorization at one place
- When different client types (mobile, web, third-party) need different API shapes

✓ Advantages	✗ Disadvantages / Limitations
+ Single entry point simplifies client-side integration	- Single point of failure if not deployed with high availability
+ Centralized cross-cutting concerns: auth, logging, rate limiting	- Can become a bottleneck under high traffic
+ Hides internal service topology from external clients	- Additional network hop adds latency
+ Protocol translation (HTTP to gRPC, WebSocket, etc.)	- Risk of becoming a 'God Gateway' with too much logic
+ Can aggregate multiple service responses into one	- Versioning and routing rules can become complex over time

Implementation Example

```
spring:
  cloud:
    gateway:
      globalcors:
        corsConfigurations:
          '[/**]':
            allowedOrigins: '*'
      default-filters:
        - name: RequestRateLimiter
          args:
```

```

redis-rate-limiter.replenishRate: 100
redis-rate-limiter.burstCapacity: 200
routes:
  - id: order-service
    uri: lb://order-service
    predicates:
      - Path=/api/v1/orders/**
    filters:
      - name: CircuitBreaker
        args:
          name: order-cb
          fallbackUri: forward:/fallback/orders
      - AddRequestHeader=X-Forwarded-Service, gateway
      - name: JwtAuthenticationFilter

```

Pattern 13 Backend for Frontend (BFF) [Communication] **Complexity: Medium**

Intent: Create a dedicated backend gateway service tailored to the needs of a specific frontend client (mobile, web, third-party partner). Each BFF aggregates data from multiple microservices and returns exactly what the client needs — no over-fetching or under-fetching.

Attribute	Details
Category	Communication
Complexity	Medium
Scope	Client-Specific API Layer
Related To	API Gateway, API Composition

When to Use

- When different clients (iOS, Android, Web, Partner API) have very different data requirements
- When mobile clients need optimized, bandwidth-efficient responses
- When you want to prevent frontend logic from leaking into backend services

 Advantages	 Disadvantages / Limitations
+ Client-optimized responses — no over/under-fetching	- Code duplication — similar aggregation logic across BFFs
+ Frontend teams own their BFF — faster iteration	- More services to maintain and deploy
+ Hides complexity of multiple backend services from clients	- Risk of BFF becoming too thick with business logic
+ Protocol flexibility — mobile BFF can use different protocol	- Cross-cutting concerns must be implemented in each BFF
+ Independent deployment of each BFF	- Increases total system complexity

Implementation Example

```
// Mobile BFF — returns compact, mobile-optimized response
@RestController
@RequestMapping("/mobile/api")
public class MobileBffController {

    @GetMapping("/dashboard")
    public MobileDashboard getDashboard(@AuthenticationPrincipal User user) {
        // Parallel fetch from multiple services
        CompletableFuture<List<Order>> orders =
            CompletableFuture.supplyAsync(() ->
                orderClient.getRecentOrders(user.getId(), 3));
        CompletableFuture<UserProfile> profile =
            CompletableFuture.supplyAsync(() ->
                userClient.getProfile(user.getId()));

        // Aggregate into mobile-optimized response
        return new MobileDashboard(
            profile.get().getDisplayName(),
            orders.get().stream()
                .map(o -> new OrderSummary(o.getId(), o.getStatus())) // compact
                .collect(toList())
        );
    }
}
```

Pattern 14 Event-Driven Architecture (Choreography)

[Communication] **Complexity: Medium**

Intent: Services communicate by publishing and subscribing to domain events via a message broker. No direct service-to-service calls — the event bus is the only coupling point. Each service reacts to events it cares about independently, creating a loosely coupled, reactive system.

Attribute	Details
Category	Communication
Complexity	Medium
Scope	Async Service Communication
Related To	Saga (Choreography), Kafka, RabbitMQ, Outbox

When to Use

- When decoupling is more important than immediate consistency
- When events naturally model your domain (order placed, payment received)
- For high-throughput, non-blocking inter-service communication

✓ Advantages	✗ Disadvantages / Limitations
+ Maximum loose coupling — services don't know about each other	- Hard to trace flow across services — requires distributed tracing
+ Highly scalable — producers/consumers	- Eventual consistency — downstream

scale independently	services lag behind
+ Event log (Kafka) provides replay capability for recovery	- Complex error handling — no immediate feedback on failures
+ New subscribers can be added without modifying producers	- Risk of event schema drift across teams
+ Natural fit for audit logging and event sourcing	- Circular event loops possible if not carefully designed

Implementation Example

```
// Order Service: publish event
@Service
public class OrderService {
    public Order placeOrder(PlaceOrderCommand cmd) {
        Order order = orderRepository.save(new Order(cmd));
        eventPublisher.publish(new OrderPlacedEvent( // fire and forget
            order.getId(), order.getItems(), order.getCustomerId()));
        return order;
    }
}

// Inventory Service: react to event
@KafkaListener(topics = "order-events", groupId = "inventory")
public void onOrderPlaced(OrderPlacedEvent event) {
    inventoryService.reserveItems(event.getItems());
    kafkaTemplate.send("inventory-events",
        new InventoryReservedEvent(event.getOrderId()));
}

// Payment Service: react independently
@KafkaListener(topics = "inventory-events", groupId = "payment")
public void onInventoryReserved(InventoryReservedEvent event) {
    paymentService.initiatePayment(event.getOrderId());
}
```

Category 5: Deployment Patterns

Pattern 15 Blue-Green Deployment [Deployment] Complexity: Low

Intent: Maintain two identical production environments (Blue = live, Green = new version). Deploy new version to Green, validate it, then switch all traffic from Blue to Green instantly. Zero downtime. Rollback is instant — just switch traffic back to Blue.

Attribute	Details
Category	Deployment
Complexity	Low
Scope	Zero-Downtime Deployment
Related To	Canary Deployment, Feature Flags
Tools	Kubernetes (service selector swap), AWS ECS, Spinnaker

When to Use

- When zero downtime is required for deployments
- When instant rollback capability is essential
- For stateless services — session affinity complications arise with stateful services

✓ Advantages	✗ Disadvantages / Limitations
+ Zero-downtime deployment — traffic switch is instantaneous	- Double infrastructure cost — running two full environments
+ Instant rollback — just re-point traffic to Blue	- Database schema changes must be backward compatible for rollback
+ Full production testing on Green before switch	- Stateful sessions may be lost on traffic switch
+ Reduces deployment risk significantly	- Not suitable for very frequent deployments (cost)
+ Simple mental model — easy for teams to understand	- Warming up the new environment (caches, JVM) before switch is needed

Implementation Example

```
# Kubernetes Blue-Green via selector swap

# Blue deployment (currently live)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service-blue
  labels:
    app: order-service
    version: blue
```

```
# Green deployment (new version, deployed but not receiving traffic)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service-green
  labels:
    app: order-service
    version: green

# Service points to blue; switch to green by updating selector:
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
    version: blue # Change to 'green' to switch traffic instantly
```

Pattern 16 Canary Deployment [Deployment] **Complexity: Medium**

Intent: Gradually route a small percentage of production traffic to the new version, monitor metrics, and progressively increase traffic if healthy. Named after canary birds used in coal mines to detect danger. Minimizes blast radius of bad deployments.

Attribute	Details
Category	Deployment
Complexity	Medium
Scope	Risk-Managed Deployment
Related To	Blue-Green, Feature Flags, Service Mesh
Tools	Istio, Argo Rollouts, Flagger, Spinnaker

When to Use

- When releasing features to a subset of users first (A/B testing)
- When new version has performance implications you want to validate
- When rolling out to millions of users and risk must be minimized

✓ Advantages	✗ Disadvantages / Limitations
+ Only a small % of users impacted if new version has issues	- Two versions serving traffic simultaneously — compatibility required
+ Real production traffic validates the new version	- More complex traffic routing setup (Istio/Envoy required)
+ Progressive confidence — increase traffic as metrics improve	- Longer deployment window than blue-green
+ Automatic rollback if error rate/latency thresholds breached	- Monitoring and alerting must be mature for automated rollback

+ Enables A/B testing with real users

- Database schema changes must be backward compatible for both versions

Implementation Example

```
# Argo Rollouts – progressive canary delivery
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: order-service
spec:
  strategy:
    canary:
      steps:
        - setWeight: 5      # 5% to canary
        - pause: {duration: 10m}
        - setWeight: 25     # 25% – monitor metrics
        - pause: {duration: 10m}
        - setWeight: 50
        - pause: {duration: 5m}
        - setWeight: 100    # Full rollout
  analysis:
    templates:
      - templateName: error-rate-check
    args:
      - name: service-name
        value: order-service
  autoPromotionEnabled: false # Manual approval for 100%
```

Pattern 17 Sidecar Pattern [Deployment] **Complexity: Medium**

Intent: Deploy a helper container (sidecar) alongside the main application container in the same Kubernetes pod. The sidecar handles cross-cutting concerns (observability, proxy, config reload, log shipping) without modifying the main application. Both containers share the same network namespace and storage.

Attribute	Details
Category	Deployment
Complexity	Medium
Scope	Cross-Cutting Concern Separation
Related To	Service Mesh, Ambassador, Adapter
Examples	Envoy proxy (Istio), Filebeat, Vault Agent, Config Watcher

When to Use

- When you want to add observability or security to services without code changes
- In service mesh deployments — Envoy sidecar provides mTLS, tracing, circuit breaking
- When deploying legacy applications that can't be modified

✔ Advantages	✘ Disadvantages / Limitations
+ Separation of concerns — app code stays clean of infrastructure logic	- Adds resource overhead — every pod has an extra container
+ Language-agnostic — sidecar works with any main application	- Sidecar lifecycle is coupled to the main container's pod
+ Consistent cross-cutting behavior across all services	- Increased latency for each network call (proxy overhead ~1ms)
+ Sidecar can be updated independently of the main app	- More complex debugging — traffic goes through proxy
+ Enables service mesh capabilities transparently	- Large-scale Envoy sidecar fleets require experienced operators

Category 6: Observability Patterns

Pattern 18 Distributed Tracing [Observability] Complexity: Medium

Intent: Track requests as they propagate across multiple microservices by assigning a unique Trace ID to each request and Span IDs for each service hop. Enables end-to-end visibility into request flows, latency bottlenecks, and error sources across the entire system.

Attribute	Details
Category	Observability
Complexity	Medium
Scope	Cross-Service Request Tracking
Related To	Log Aggregation, Metrics, Service Mesh
Tools	Zipkin, Jaeger, AWS X-Ray, Datadog APM, OpenTelemetry

When to Use

- In all microservices systems — essential for debugging distributed request flows
- When diagnosing performance issues spanning multiple services
- When correlating logs across services for a single user request

✓ Advantages	✗ Disadvantages / Limitations
+ End-to-end request visibility across all services	- 100% sampling at high traffic is expensive — use sampling strategies
+ Pinpoint latency bottlenecks to specific services or operations	- Trace context propagation must be implemented across all services
+ Correlate logs across services using Trace ID	- Storage costs for trace data at scale
+ Visualize service dependency graphs from actual traffic	- Async flows (Kafka) require explicit trace context propagation
+ OpenTelemetry standard — vendor-neutral instrumentation	- Requires mature tooling (Jaeger/Zipkin) for full value

Implementation Example

```
# Spring Boot 3 + Micrometer Tracing + Zipkin

# pom.xml
# io.micrometer:micrometer-tracing-bridge-brave
# io.zipkin.reporter2:zipkin-reporter-brave

# application.yml
management:
  tracing:
    sampling:
      probability: 0.1 # 10% in prod — 100% in dev
```

```
spring:
  zipkin:
    base-url: http://zipkin:9411

// Trace ID automatically injected in all log lines:
// 2024-01-01 [order-service,,trace=abc123,span=def456] Processing order

// Propagating trace context to Kafka messages:
@Bean
public ProducerFactory<String, Object> producerFactory(
    ObservationRegistry registry) {
    // Micrometer auto-propagates trace context to Kafka headers
    return new DefaultKafkaProducerFactory<>(configs, registry);
}
```

Pattern 19 Health Check API [Observability] **Complexity: Low**

Intent: Each microservice exposes health check endpoints that infrastructure (Kubernetes, load balancers, service registries) can probe to determine if the service is alive and ready to receive traffic. Spring Boot Actuator provides this out of the box.

Attribute	Details
Category	Observability
Complexity	Low
Scope	Service Health Verification
Related To	Service Discovery, Kubernetes Probes, Load Balancing
Endpoints	/actuator/health/liveness /actuator/health/readiness

 When to Use

- Standard pattern — implement in every microservice
- Used by Kubernetes liveness and readiness probes
- Used by Eureka / Consul to determine service health
- Used by load balancers to route traffic only to healthy instances

✔ Advantages	✖ Disadvantages / Limitations
+ Automatic detection and recovery from unhealthy instances	- Poorly implemented checks can cause unnecessary restarts
+ Spring Boot Actuator provides out-of-the-box health checks	- Deep health checks (DB queries) add load to the database
+ Customizable — add business-level health indicators	- Distinguishing liveness vs readiness requires careful design
+ Deep health checks validate DB, cache, and external dependency health	- Health endpoint itself must be always available — even under load
+ Enables zero-downtime deployments via readiness probe	- Too broad health checks cause cascading failures when dependencies fail

Implementation Example

```
# application.yml
management:
  endpoint:
    health:
      show-details: always
      probes:
        enabled: true    # /health/liveness + /health/readiness
  health:
    db:
      enabled: true
    redis:
      enabled: true

// Custom health indicator
@Component
public class PaymentGatewayHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        try {
            if (paymentGateway.ping()) {
                return Health.up()
                    .withDetail("gateway", "reachable")
                    .withDetail("latency", "12ms")
                    .build();
            }
        } catch (Exception e) {
            return Health.down()
                .withDetail("error", e.getMessage())
                .build();
        }
        return Health.unknown().build();
    }
}
```

Pattern 20 Log Aggregation [Observability] Complexity: Low

Intent: Centrally collect, store, and index logs from all microservice instances. Since a single request touches multiple services, distributed log aggregation with correlation IDs enables tracing a complete transaction across all services in one query.

Attribute	Details
Category	Observability
Complexity	Low
Scope	Centralized Logging
Related To	Distributed Tracing, Metrics
Stacks	ELK (Elasticsearch + Logstash + Kibana) Loki + Promtail + Grafana

When to Use

- Standard practice — implement in all microservices environments
- Mandatory when running 3+ microservices in production
- Essential for debugging cross-service issues

✔ Advantages	✖ Disadvantages / Limitations
+ Single pane of glass — all service logs in one place	- ELK stack is resource-intensive — significant infrastructure cost
+ Correlation ID enables tracing one request across 10 services	- Log volume at scale requires careful retention policy
+ Structured JSON logs enable powerful filtering and alerting	- Sensitive data in logs requires masking/filtering
+ Log retention and archiving managed centrally	- Developers must discipline themselves to use structured logging
+ Real-time alerting on error patterns (Kibana/Grafana alerts)	- Centralized log store becomes a single point of failure

 Implementation Example

```
# application.yml - Structured JSON logging with correlation ID
logging:
  pattern:
    console: "%d{ISO8601} %X{traceId} %X{spanId} %-5level %logger - %msg%n"

# Log output includes trace ID from Micrometer Tracing:
# 2024-01-01T10:00:00 abc123 def456 INFO OrderService - Order placed
# 2024-01-01T10:00:01 abc123 789abc INFO PaymentService - Payment charged
# Same traceId (abc123) → single query shows complete flow

// Logback + logstash-logback-encoder for JSON output to ELK
// logback-spring.xml
// <appender name='LOGSTASH'>
//   <destination>logstash:5000</destination>
//   <encoder class='net.logstash.logback.encoder.LogstashEncoder' />
// </appender>
```

Patterns Quick-Reference Summary

All 20 patterns at a glance — Category, Complexity, Primary Problem Solved, and Key Tools.

#	Pattern	Complexity	Problem Solved	Key Tools / Technologies
1	Decompose by Business Capability	Medium	Monolith splitting	DDD, Conway's Law
2	Decompose by Subdomain (DDD)	High	Fine-grained boundaries	Bounded Context, ACL
3	Strangler Fig	Medium	Incremental migration	API Gateway
4	Database per Service	Medium	Data isolation & autonomy	PostgreSQL, MongoDB, Redis
5	Saga Pattern	High	Distributed transactions	Axon, Kafka, RabbitMQ
6	CQRS	High	Read/write optimization	Axon, Kafka, Elasticsearch
7	Event Sourcing	High	Audit trail & state replay	EventStore, Axon
8	Transactional Outbox	Medium	Guaranteed event publishing	Debezium CDC, Kafka
9	Circuit Breaker	Medium	Cascading failure prevention	Resilience4j, Hystrix
10	Bulkhead	Medium	Resource isolation	Resilience4j ThreadPool
11	Retry + Backoff	Low	Transient fault handling	Resilience4j, Spring Retry
12	API Gateway	Medium	Single entry point	Spring Cloud Gateway, Kong
13	Backend for Frontend (BFF)	Medium	Client-optimized APIs	Spring Boot, GraphQL
14	Event-Driven (Choreography)	Medium	Async loose coupling	Kafka, RabbitMQ, SNS/SQS
15	Blue-Green Deployment	Low	Zero-downtime deployment	K8s, Spinnaker, AWS
16	Canary Deployment	Medium	Risk-managed rollout	Argo Rollouts, Flagger, Istio
17	Sidecar	Medium	Cross-cutting concerns	Envoy, Filebeat, Vault Agent
18	Distributed Tracing	Medium	Cross-service request tracking	Zipkin, Jaeger, OpenTelemetry
19	Health Check API	Low	Infrastructure health monitoring	Spring Actuator, Kubernetes
20	Log Aggregation	Low	Centralized log management	ELK Stack, Loki + Grafana

Pattern Combination Recommendations

Patterns work best in combination. Here are the most effective groupings:

Use Case	Recommended Pattern Combination
Distributed transactions	Saga + Outbox + Event Sourcing + Idempotent Consumers
High availability service calls	Circuit Breaker + Retry (backoff) + Bulkhead + Fallback
Event-driven data flow	Event-Driven + CQRS + Outbox + Log Aggregation
Safe service releases	Blue-Green or Canary + Health Check + Distributed Tracing
Monolith migration	Strangler Fig + API Gateway + Anti-Corruption Layer
Cross-service observability	Distributed Tracing + Log Aggregation + Health Check API
Secure service mesh	Sidecar (Envoy) + mTLS + API Gateway + BFF

Master these patterns and you master microservices architecture.

No pattern is a silver bullet — always evaluate trade-offs in your specific context.